

IFSP • ADS • JAVASCRIPT • 24 DE AGOSTO DE 2026 • NOTURNO • 5 AULAS

# Do *objeto* à herança, e da herança à *qualidade* do código.

Como o JavaScript representa dados na memória, como objetos herdam comportamento por protótipos, e como manter o código limpo.

**Continuando a Aula 03.** Cada exemplo aqui aplica as boas práticas que vimos: nomes autoexplicativos, `PascalCase` / `camelCase` / `UPPER_SNAKE_CASE`, responsabilidade única, encapsulamento com `#`, funções puras, *guard clauses* e feedback claro de erro. Código como interface para outro humano.

Tópico 1 • Tipos primitivos e de referência

Tópico 2 • Construtores, protótipos e herança

Tópico 3 • Code smells e linters

## // ROTEIRO DA NOITE

Tópico 1

### Modelo de Objetos

Primitivos × referência, métodos e propriedades.

Tópico 2

### Herança por Prototipação

Composição, construtores, protótipos e cadeia.

Tópico 3

### Qualidade de Código

Code smells, refatoração e linters (ESLint).

Laboratório

### Prática integradora

Desafio integrador + quiz de revisão.

01

# O Modelo de Objetos em JavaScript

tipos primitivos e de referência • métodos e propriedades dos objetos

Tudo em JavaScript é ou um **valor primitivo** ou um **objeto**. Essa distinção explica quase todos os comportamentos "estranhos" do dia a dia: cópia, comparação, mutação e passagem de argumentos.

## Os dois grupos de tipos // classificação

| PRIMITIVOS (POR VALOR)                                                                                                                                       | REFERÊNCIA (POR REFERÊNCIA)                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>string</code> , <code>number</code> , <code>boolean</code> , <code>null</code> ,<br><code>undefined</code> , <code>symbol</code> , <code>bigint</code> | <code>object</code> , <code>array</code> , <code>function</code> , <code>date</code> , <code>map</code> ,<br><code>set</code> ... |
| Imutáveis. A variável guarda <b>o valor em si</b> .                                                                                                          | Mutáveis. A variável guarda <b>um endereço</b> (o objeto vive no heap).                                                           |
| Copiar = duplicar o valor. Uma cópia não afeta a outra.                                                                                                      | Copiar = duplicar o endereço. Duas variáveis apontam para o <b>mesmo</b> objeto.                                                  |

IDEIA CENTRAL

Primitivo é *coisa*; objeto é *onde a coisa mora*. Ao atribuir um objeto a outra variável, você copia o mapa, não a casa.

## Valor × referência ao vivo // execute e mostre

Nomes descritivos, `const` por padrão e uma constante nomeada no lugar de número mágico — como na Aula 03.

script.js   BOA PRÁTICA   reset   ▶ run

```
// Primitivos: cópia independente
let notaOriginal = 10;
let notaCopia = notaOriginal; // copia o VALOR
notaCopia = 9;
console.log("original:". notaOriginal. "| copia:". notaCopia):
```

## Aula 04 • Objetos & Herança em JS

[agenda](#) [objetos](#) [herança](#) [qualidade](#) [lab](#)

### PEGADINHA CLÁSSICA

`{ x: 1 } === { x: 1 }` é **false**. A comparação verifica se as variáveis apontam para o mesmo lugar, não se o conteúdo é parecido.

## Passagem de argumentos // função pura vs. mutação

A Aula 03 recomenda **funções puras**: mesmo input, mesmo output, sem efeitos colaterais. Compare a versão que muta o objeto de fora (efeito colateral escondido) com a versão pura que devolve um novo objeto.

script.js

BOA PRÁTICA

reset

▶ run

```
const PERCENTUAL_DESCONTO = 0.1;

// Efeito colateral: altera o objeto recebido (evite quando não for intencional)
function aplicarDescontoMutando(produto) {
  produto.preco *= 1 - PERCENTUAL_DESCONTO;
}

// clique em run para executar
```

## Copiando objetos de verdade // shallow x deep

Spread e `Object.assign` fazem cópia **rasa**. Objetos aninhados continuam compartilhados.

script.js

BOA PRÁTICA

reset

▶ run

```
const alunoBase = { nome: "Ana", notas: [8, 9] };

// Cópia rasa: primeiro nível é independente...
const alunoCopiaRasa = { ...alunoBase };
alunoCopiaRasa.nome = "Bruno";

// clique em run para executar
```

## Métodos e propriedades // anatomia + encapsulamento

## Aula 04 • Objetos &amp; Herança em JS

agenda objetos herança qualidade lab

script.js

BOA PRÁTICA

reset

▶ run

```
class ContaBancaria {  
  #saldo; // campo privado protege o estado interno  
  
  constructor(titular, saldoInicial) {  
    this.titular = titular;  
  }  
  
  // clique em run para executar
```

## IHC NO CÓDIGO

O `#saldo` privado é como uma interface que não expõe controles perigosos; a validação é prevenção de erro; a mensagem no `throw` é feedback claro. Código legível é usabilidade para o próximo dev.

## EXERCÍCIO RÁPIDO – 5 MIN

Crie uma classe `Carrinho` com um campo privado `#itens` (array), um método `adicionarItem({ nome, preco })` e um getter `total`. Use nomes verbais e parâmetro como objeto, como a Aula 03 recomenda.

▶ ver solução

02

## Composição, Construtores e Herança por Prototipação

composição • construtores • protótipos • herança por prototipação

JavaScript não nasceu com classes tradicionais. O mecanismo real de reuso é a **cadeia de protótipos**: todo objeto tem um link para outro objeto (seu protótipo), e quando uma propriedade não é encontrada, a busca sobe por essa cadeia.

## Aula 04 • Objetos & Herança em JS

[agenda](#) [objetos](#) [herança](#) [qualidade](#) [lab](#)

| COMPOSIÇÃO ("TEM UM")                                                | HERANÇA ("É UM")                                            |
|----------------------------------------------------------------------|-------------------------------------------------------------|
| Um objeto <b>contém</b> outros e delega tarefas a eles.              | Um objeto <b>deriva</b> de outro pela cadeia de protótipos. |
| Flexível, evita hierarquias rígidas. "Um carro <b>tem</b> um motor." | Boa para especialização. "Um gato <b>é</b> um animal."      |

script.js BOA PRÁTICA reset ▶ run

```
// COMPOSIÇÃO: o carro tem um motor e delega a ele
const motor = {
  ligar() {
    return "vrum!";
  },
};

// clique em run para executar
```

## Funções construtoras // o padrão clássico

Construtora é uma função chamada com `new`. Note o nome em **PascalCase** (é um "molde", como uma classe) e o método no protótipo, não copiado por instância.

script.js BOA PRÁTICA reset ▶ run

```
function Aluno(nome, curso) {
  this.nome = nome; // propriedade de instância (única por objeto)
  this.curso = curso;
}

// clique em run para executar
```

### POR QUE NO PROTÓTIPO?

No protótipo existe **uma** função para todas as instâncias — economia de memória e ponto único de manutenção. Dentro do construtor, cada objeto teria sua própria cópia.

## Aula 04 • Objetos & Herança em JS

[agenda](#) [objetos](#) [herança](#) [qualidade](#) [lab](#)

script.js

BOA PRÁTICA

reset

▶ run

```
function Aluno(nome) {  
  this.nome = nome;  
}  
  
Aluno.prototype.saudar = function () {  
  
// clique em run para executar
```

## Herança por prototipação // especializar

Ligamos o protótipo do "filho" ao do "pai". Repare no reúso via `super` / `.call` — a Aula 03 pede "estenda em vez de reescrever" e mantenha a hierarquia rasa.

script.js

BOA PRÁTICA

reset

▶ run

```
function Aluno(nome) {  
  this.nome = nome;  
}  
  
Aluno.prototype.apresentar = function () {  
  
// clique em run para executar
```

## A sintaxe moderna: `class` // açúcar sintático

A palavra `class` (ES6) é açúcar sobre esse mesmo mecanismo. Aqui reproduzimos o exemplo `Usuario` / `UsuarioAdmin` da Aula 03, com **PascalCase**, `super()` e hierarquia rasa.

script.js

BOA PRÁTICA

reset

▶ run

```
class Usuario {  
  #senha; // campo privado – protege o estado interno  
  
  constructor(nome, email) {  
    this.nome = nome;  
  
// clique em run para executar
```

## Aula 04 • Objetos &amp; Herança em JS

agenda objetos herança qualidade lab

Mostre que `typeof User.isAdmin === 'function'` mesmo sendo como `class`. Classe em JS não é um tipo novo — é uma função construtora com uma cara mais amigável e mais próxima das convenções da Aula 03.

## EXERCÍCIO RÁPIDO — 8 MIN

Crie uma classe `Forma` com método `calcularArea()` retornando 0, e uma subclasse `Retangulo` que recebe `{ base, altura }` e sobrescreve `calcularArea()`. Use PascalCase, nome verbal no método e parâmetro como objeto.

[▶ ver solução](#)

03

## Qualidade de Código: Code Smells e Linters

code smells • linters (ESLint) • refatoração

Código que "funciona" não é necessariamente bom. **Code smells** são sinais na superfície que apontam um problema de design. Eles são exatamente o oposto das boas práticas da Aula 03 — e é assim que vamos organizá-los.

## Cheiros comuns // e a regra da Aula 03 que ferem

## Nomes ruins

Fere *affordance/clareza*: nomes como `x`, `data2` não comunicam intenção.

## Função longa

Fere *responsabilidade única*: faz coisas demais, difícil de testar.

## Código duplicado

Fere manutenção: uma correção precisa ser feita em N lugares.

## Números mágicos

Fere clareza: valores soltos sem constante nomeada que explique o porquê.

## Aninhamento profundo

## Muitos parâmetros

## Aula 04 • Objetos & Herança em JS

[agenda](#) [objetos](#) [herança](#) [qualidade](#) [lab](#)

### Refatoração ao vivo // antes → depois

O código abaixo funciona, mas concentra vários smells: nome vago, número mágico, comparação frágil `==` e duplicação. Rode e observe a saída.

script.js EVITE reset ▶ run

```
// ANTES – funciona, mas cheira mal
function calc(l) {
  let t = 0;
  for (let i = 0; i < l.length; i++) {
    if (l[i].tipo == "livro") {

// clique em run para executar
```

Agora a versão refatorada seguindo a Aula 03: constante nomeada, função pura pequena, nomes verbais, `===` e sem duplicação. Mesma saída, muito mais legível.

script.js BOA PRÁTICA reset ▶ run

```
// DEPOIS – nomes claros, sem número mágico, sem duplicação
const PERCENTUAL_DESCONTO_LIVRO = 0.1;

function calcularPrecoItem(item) {
  const islivro = item.tipo === "livro":

// clique em run para executar
```

### Guard clauses contra aninhamento // retorno antecipado

Direto da Aula 03: retornos antecipados achatam a lógica e reduzem a carga cognitiva de quem lê.

script.js BOA PRÁTICA reset ▶ run

## Aula 04 • Objetos & Herança em JS

[agenda](#) [objetos](#) [herança](#) [qualidade](#) [lab](#)

```
if (!pedido) return null;  
if (pedido.itens.length === 0) return null;
```

// clique em run para executar

## Linters: automatizando a vigilância // ESLint + Prettier

Um **linter** analisa o código *sem executá-lo* e aponta problemas. Em JS, o padrão é o **ESLint**; junto do **Prettier** (formatação), eles funcionam como o "design system do código" que a Aula 03 descreve.

- Detecta comparações frágeis (`==` vs `===`), variáveis não usadas e `console.log` esquecido.
- Padroniza estilo: 2 espaços de indentação, ponto e vírgula, aspas — sem esforço manual.
- Integra ao editor (sublinha na hora) e à CI, barrando código problemático antes do merge.

```
// eslint.config.js – algumas regras que refletem a Aula 03  
export default [  
  {  
    rules: {  
      "eqeqeq": "error",          // exige === (comparação previsível)  
      "no-unused-vars": "warn",   // avisa variáveis não usadas  
      "no-magic-numbers": "warn", // alerta números mágicos  
      "camelcase": "error",       // impõe a convenção de nomes  
    },  
  },  
];
```

### LINTER × FORMATTER

**ESLint** cuida de qualidade e prováveis erros; **Prettier** cuida só da aparência.

Comentários explicam o *porquê*, não o *quê* — o código bem nomeado já diz o que faz.

### Qual destes é um **code smell**, e não um erro de sintaxe?

Esquecer de fechar um parêntese

**Aula 04 • Objetos & Herança em JS**

agenda objetos herança qualidade lab

Chamar uma variável antes de declará-la com `let`

04

## Laboratório integrador

objetos + herança + qualidade numa tacada só

Desafio para fazer junto ou em duplas: usar herança por prototipação e escrever código limpo, aplicando as convenções da Aula 03.

### DESAFIO

Modele um pequeno sistema de biblioteca:

- Classe `ItemAcervo` (PascalCase) com `titulo` e método `gerarDescricao()`.
- Subclasse `Livro` (com `autor`) que usa `super` e sobreescreve `gerarDescricao()`.
- Subclasse `Revista` (com `edicao`) idem.
- Um array misto e um `forEach` que imprime cada descrição (polimorfismo!).